



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۲۷ مهر

فصل دوم: لایه Application اسلاید ۳۳ تا ۵۸ [۱]

۴ مولفه cookieها

- یک خط به عنوان *cookie header line* در هدر *http response* دارد.
- در *request* بعدی *client* هم، در *header line* هایش، یک *cookie header line* اضافه می‌شود.
- مورد سوم اینکه باید اطلاعات آن در *browser* ذخیره شود.
- مولفه چهارم هم اینک اطلاعات در وب سرور هم باید ذخیره شود. *backend database*
- درواقع وقتی کاربر از وب سرور برای اولین بار درخواست می‌کند، سرور برای اون شخص، یک *id* منحصر به فرد برای او و ثانیاً یک *entry* در دیتابیس *backend* برای آن *id* ایجاد می‌کند.
- درخواست های *http* بعدی فرد، حاوی مقدار *id* کوکی است و باعث می‌شود هویتش را تشخیص دهد. (در واقع در اولین پاسخ سرور، *set cookie* مثلاً مساوی ۱۶۷۸ اضافه می‌شود. و در دفعات بعدی *request* یک خط مربوط به $cookie = 1688$ اضافه شده که همان *id* است.)
- درواقع *browser* این عبارت $cookie = 1678$ را اضافه می‌کند.
- هر بار هم در سمت سرور با طبق اون کوکی، عملیات مربوط به آن را در سرور بروز می‌کند.

کاربرد کوکی‌ها

- تصدیق اصالت سبک
- برای خرید اینترنتی و در طول زمان خرید استفاده می‌شود
- پیشنهاد دادن طبق اطلاعات مشتری
- وضعیت *session* یا نشست را نگه می‌دارد. کاربرد در *web email*ها یا ایمیل های مبتنی بر وب.
- *state* ها هم در سمت *client* هم در سمت سرور در نقاط انتهایی ذخیره می‌شود.
- دوماً، در خود پیغام‌ها هم ذخیره می‌شود. سمت مقابل وضعیت ما را از همین پیغام‌ها دریافت می‌کند

کوکی‌ها اجازه می‌دهند که سایت‌ها اطلاعات زیادی از شما بدانند، مثل نام و ایمیل و غیره. و در طول زمان این اطلاعات بیشتر می‌شوند.

*third party*ها می‌توانند این کوکی‌ها (و هویت‌ها) را ردگیری کنند.

web caches

اگر این قابلیت را در *browser* فعال کنیم، برورر اول از *proxy server*، آن *http request* را درخواست می‌کند. اگر در پروکسی سرور، (یا همان *web cache*) اطلاعات آن موجود بود، برمی‌گرداند وگرنه آن *request* را به سرور اصلی می‌فرستد و *response* را دریافت می‌کند. بعد جوابی را که گرفته *cache* می‌کند و بعدش به کاربر می‌دهد.

این مکانیسم باعث می‌شود که بتوان پاسخ‌های *http* را سریع‌تر داد.

درواقع به عنوان سرور برای مشتری عمل می‌کند و درمقابل سرور وب، نقش *client* را می‌گیرد.

سرور در پاسخ به *cache*، از مکانیسم *cache control* استفاده می‌کند تا تنظیمات مربوط به کش و نحوه کش کردن را به *cache* یاد می‌دهد. مثلاً اینکه حداکثر عمر *object* چقدر باشد یا کدام *object*ها کش نشوند.

از *web cache*ها در دانشگاه‌ها و *ISP*ها و شرکت‌ها و نواحی مسکونی، نصب می‌شوند

فواید

- زمان پاسخ برای درخواست مشتری‌ها را کاهش می‌دهد چون *cache* به *client* نزدیک‌تر است.
 - ترافیک روی لینک دستیابی موسسات را کاهش می‌دهد.
 - اجازه می‌دهد که *ISP* یا *content provider*های ضعیف‌تر، بتوانند محتوای خود را بهتر ارائه کنند.
- یک مثال از محاسبه *performance* و *link utilization* و همچنین محاسبه تاخیر *end to end* در اسلاید‌ها، با و بدون داشتن *web caching* گفته شد.
- وقتی از *web cache* استفاده نکردیم، به دلیل اینکه *utilization* اون *access link* بین روتر محلی دانشگاه و روتر مرزی دانشگاه بالا بود (نزدیک 0.97 بود چون کلاً می‌توانست 1.54Mbps عبور دهد و الان هم داشت به طور میانگین
- $$15(\text{request per second}) \times 100\text{kb}(\text{web object size}) = 1.5\text{Mbps}$$

$$\text{access link utilization} = \frac{1.5\text{Mbps}}{1.5\text{Mbps}} = 0.97$$

و همانطور که قبل گفتیم، اگر این مقدار کمتر از 0.7 باشد، تاخیر *queueing* کم است اما اگر بالای 0.7 و نزدیک به ۱ باشد، زیاد است. اینجا مثلاً فقط چند دقیقه همین تاخیر می‌شود.

پس دو راه داریم. یا *access link* را بهتر کنیم و *bandwidth* آن افزایش یابد (مخرج کسر زیاد می‌شود و آن عدد پایین می‌آید). که هزینه زیادی دارد، یا اینکه در همان *LAN* خودمان، یک *web cache* قرار دهیم. اگر فرض کنیم

که ۴۰ درصد درخواست‌هایمان به طور میانگین توسط آن *web cache* پاسخ می‌گیرد، پس آن ۴۰ درصد لازم نیست تاخیر رفت و برگشت روی اینترنت را داشته باشند و در چند میلی ثانیه در همان *LAN* پاسخ را دریافت می‌کنند. از طرفی، تنها ۶۰ درصد باید از طریق اینترنت عمومی وصل شوند. پس بهره‌وری *access link* برابر خواهد بود با:

$$access\ link\ utilization = \frac{0.6 \times 1.5Mbps}{1.54Mbps} = 0.58\ Mean$$

که این عدد کمتر از ۰.۷ است، پس تاخیر آن ۶۰ درصد بسته فقط بیشتر فقط تاخیر رفت و برگشت *E2E* در اینترنت است که در مثال ما ۲ ثانیه بود. (بقیه تاخیرها در حد چند میلی ثانیه است) پس میانگین تاخیر کل بسته‌ها برابرست با:

$$0.6 \times 2 + 0.4 \times 0 = 1.2\ sec$$

پس میانگین تاخیر ۱.۲ ثانیه شد که هم نسبت به حالت استفاده از لینک *access* قوی‌تر، که ۲ ثانیه بود، کمتر است، و هم هزینه‌اش از آن خیلی خیلی کمتر است. حال چگونه متوجه می‌شویم که می‌توانیم از *cache* استفاده کنیم و لازم نیست سراغ سرور اصلی برویم؟ برای اینکار باید از نسخه بهبود یافته *GET* استفاده کنیم.

Conditional GET

در این روش اگر دیتای کش شده باشد و *up – to – date* باشد، *web cache* خودش جواب کاربر را می‌دهد. فرض کنید که یک درخواستی به *web cache* آمده. حال *web cache* در نقش *client* به سرور اصلی *request* می‌دهد و در *header line* آن یک فیلد با عنوان *< date > :if – modified – since* می‌آورد. سرور بعد از دریافت آن، چک می‌کند که آیا دیتا بعد از تاریخ گفته شده تغییر کرده یا نه. اگر تغییر نکرده باشد، یک *response* کوچک با این *status line* می‌فرستد که *HTTP/1.0 304 Not Modified* و *web cache* متوجه می‌شود که دیتایی که دارد، آپدیت نشده و همان را می‌فرستد. اما اگر سرور پاسخ دهد که *HTTP/1.0 200 OK*، یعنی دیتا آپدیت شده و به همراه اون *status line*، خود دیتای آپدیت شده را هم در بدنه پاسخ، به *web cache* می‌فرستد.

HTTP2

در *http1.1* از درخواست‌های *Multiple GET* و قابلیت *pipeline* پشتیبانی می‌کند. و درخواست‌ها را به صورت *FCFS = first in first served* انجام می‌دهد. مشکلش این است که درخواست‌ها (آبجکت)ی کوچک پشت درخواست‌ها (آبجکت)ی بزرگ بیفتد. این اتفاق معروف به *HOL blocking = head of line blocking* است (بلاکینگ یا انسداد سر خط)

علاوه بر این، مکانیسم *loss recovery* هم که برای *retransmit* کردن داده‌های دراپ شده بود، باعث توقف انتقال *object*‌ها می‌شود (چرا واقعا؟ متوجه نشدم). هدف ما این است که هزینه *multi-object* رو کم کنیم. در *HTTP2* ارسال آبجکت‌ها از سرور به مشتری‌ها، انعطاف پذیر تر شده است. درواقع ترتیب اولویت‌ها لزوماً *FCFS* نیست و طبق تقدم *priority*‌هایی که مشتری می‌خواهد، مشخص می‌شود. پیغام‌ها دارای *priority*‌های مختلف هستند. در *HTTP2* با فریم بندی کردن آبجکت‌ها و زمان بندی ارسال آنها، سعی می‌کند مشکل *HOL* را تا حدی کمتر کند. (در واقع از هر فایل یک فریم به ترتیب می‌فرستد و در مثال اسلاید، این کار ارسال فریم از هر فایل را به صورت *round robin* انجام داده). در *HTTP2* ممکن است داده‌هایی که کاربر درخواست نکرده هم برای کاربر ارسال شود.

HTTP3

http2 روی یک اتصال *TCP* منفرد برقرار می‌شود و اگر یک *packet loss* رخ دهد، برای ترمیم بسته‌های تلف شده، کل ارسال متوقف می‌شود (چون کل ارسال بسته‌ها تا زمانی که آن بسته *drop* شده، *retransmit* شود، باید متوقف شوند). برای همین *browser*‌ها سعی می‌کنند با *http1.1* چند تا اتصال *TCP* برقرار کنند تا از این مشکل گفته شده (توقف یا *stall*) جلوگیری شود و گذردهی شان هم افزایش پیدا کند. مشکل *HTTP2* این است که هنوز برای برای اتصال *TCP* خام یا *vanilla*، امنیت ندارد. این مورد امنیت به *HTTP3* اضافه شده است و از پروتکل *UDP* استفاده می‌کند، کارهای جانبی و چک‌های بیشتری مثل *error checking* و *congestion control* انجام می‌دهد و قابلیت *pipelining* بهتری هم دارد.

ایمیل

ایمیل، ۳ بخش اصلی دارد:

- (۱) *user agents* که در واقع، همان نرم افزار *mail reader* است که می‌توان در آن، ایمیل را ایجاد، ادیت و مطالعه کرد. از انواع این نرم افزارها می‌توان به *outlook* و *Gmail* اشاره کرد.
- (۲) *MTA = message transfer agents* یا *mail server* وظیفه انتقال ایمیل بین *compute node*‌ها را دارند. در واقع هم در مبدا و هم مقصد، سرور ایمیل است. از نرم افزارهای سرور ایمیل می‌توان به *Postfix* و *Exim* اشاره کرد که *open source* هستند.

(۳) $SMTP = simple\ mail\ transfer\ protocol$ که درواقع پروتکل مربوط به ایمیل است.

در مورد سرور ایمیل و نرم افزار های آن مثل *Posix*، این سرور ایمیل نسبت به *client* فرستنده ایمیل، نقش سرور را ایفا می‌کند و به عنوان یک لایه امنیتی عمل می‌کند و از اینکه *client* مورد حمله *malware* یا بدافزار ها قرار بگیرد و یا *spambot* یا اپ هایی که به صورت خودکار، *spam* تولید می‌کنند، به آن حمله کند، جلوگیری می‌کند. همچنین همین *Posix* با ترکیب با بقیه نرم افزار ها می‌تواند کارهای بیشتری روی این ایمیل ها انجام دهد. مثلاً فیلتر کردن اسپم یا ویروس ها و غیره.

خود همین سرور ایمیل، می‌تواند نقش *client* گرفته و وظیفه ارسال ایمیل به سرور ایمیل مقصد را بر عهده داشته باشد. این کار را با *parallelism* انجام می‌دهد که کارایی بسیار بالایی دارد. از نرم افزار های دیگر مثل نرم افزارهای *mailing – list* هم کمک می‌گیرد.

به سرور ایمیل، *SMTP server* نیز می‌گویند.

ایمیل های دریافتی (*incoming*) در *mailbox* سرور ایمیل قرار می‌گیرد. ایمیل های ارسال شده (*outgoing*) هم در *message queue* در سرور ایمیل قرار می‌گیرد.

در واقع فرض کنید آلیس می‌خواهد با باب ایمیل بفرستد. آلیس در *mail – reader* یک ایمیل ایجاد می‌کند. این ایمیل را (به کمک پروتکل *SMTP*) به سرور ایمیل مبدا خودش می‌فرستد. این ایمیل چون *outgoing* است در *message queue* سرور ایمیل قرار می‌گیرد. بعد سرور ایمیل مبدا یک اتصال *TCP* با سرور ایمیل مقصد برقرار کرده و ایمیل را برای او می‌فرستد. بعد ایمیل در *mailbox* سرور ایمیل مقصد قرار گرفته است. باب می‌تواند از طریق *mail – reader* خودش، ایمیل را بخواند.

درواقع پروتکل *SMTP* برای انتقال ایمیل به صورت مطمئن از *TCP* در لایه *transport* استفاده می‌کند. اتصال بین سرور ایمیل مبدا و مقصد به صورت مستقیم و *direct transfer* است یعنی هیچ سرور میانی برای انتقال ایمیل بین آنها استفاده نمی‌شود.

client SMTP server یا همان سرور ایمیل مبدا، یک اتصال *TCP* به پورت ۲۵ سرور (که مربوط به ایمیل است) برقرار می‌کند. بعد از برقراری این اتصال *TCP*، طبق شکل اسلاید و طبق متن، ۳ فاز پروتکل *SMTP* انجام می‌شود.

- اول یک *SMTP handshake* بین مبدا و مقصد رخ می‌دهد.

- دوم *SMTP transfer of data* یا انتقال داده رخ می‌دهد.

- سوم *SMTP closure* رخ می‌دهد و اتصال *SMTP* قطع می‌شود.

درواقع *SMTP* تقریباً مثل *HTTP* عمل می‌کند و اینجا *command* ارسال شده و *response* دریافت می‌شود.

هم *command* و هم *response* به صورت کد *ASCII* (۷ بیتی) هستند (مثل *HTTP*)

اینجا هم *response*، دارای *status code and phrase* است.

مثال

مثالی از نحوه ارتباط بین سرور ایمیل مبدا و مقصد پس از برقراری اتصال *TCP* در اسلاید ۵۵ آمده است. کلیات آن به این شکل است که ابتدا آدرس سرور ایمیل خود را برای یکدیگر می‌فرستند و اتصال *SMTP* با سلام و علیک و کد ۲۲۰، برقرار می‌شود.

سپس سرور ایمیل *client*، آدرس ایمیل فرستنده و آدرس ایمیل گیرنده را برای سرور ایمیل مقصد ارسال می‌کند و سرور ایمیل مقصد هم کد ۲۵۰ به معنای تایید را می‌فرستند.

بعد سرور ایمیل *client*، پیام *DATA* را می‌فرستند و سرور ایمیل مقصد با کد ۳۵۴ درخواست می‌کند که خط به خط برایش ایمیل را بفرستند و بعد یک نقطه خالی بفرستند که نشان دهد پیام تمام شده.

بعد از دریافت خطوط و نقطه، سرور کد ۲۵۰ می‌فرستد که یعنی ایمیل را دریافت کرده و بعد *client* پیغام *QUIT* می‌فرستد و سرور هم با کد ۲۲۱ به آن خاتمه می‌دهد.

تفاوت *SMTP* با *HTTP*

اینکه پروتکل *HTTP* یک پروتکل *client pull* است و داده را از سرور به کلاینت می‌کشیم. اما پروتکل *SMTP*، یک پروتکل *client push* است و داده ایمیل را از *client* به سرور می‌فرستیم. هر دو به صورت *command* و *response* هستند و به صورت کد *ASCII* هستند و *Status code* دارند. در *SMTP* قابلیت *multi – part* وجود دارد که می‌توان چند تا *object* را در یک پیام بفرستد اما در *HTTP* هر *object* باید در یک پیام *response* جداگانه باشد. (البته سرچ کردم توی جمینای، میگفت توی *http* هم *multipart* داره)

اتصال *SMTP* باید *persistent* باشد.

در *SMTP* هم *body* هم *header* باید *ASCII* ۷ بیتی باشد. همانطور هم که گفتیم. پایان متن آن باید به صورت *CRLF.CRLF* باشد. (همان یک نقطه خالی در یک خط) ($CRLF = carriage\ return\ line\ feed = \backslash r \backslash n$)

قالب پیام

خود قالب پیام هم می‌تواند پروتکل داشته باشد. مثلاً در *RFC 2822*، قالب انتقال پیام مشخص شده. که باید یک *to* و یک *from* داشته باشد (الزامی) و یک *subject* (اختیاری).

بعد از هدر و قبل از *body* یک خط فاصله می‌آید. بدنه می‌تواند پیغامی با متن از کی باشد.

IMAP & web mails

همانطور که گفتیم، پیام *user agent* با پروتکل *SMTP* به سرور ایمیل مبدا می‌رود و با همان پروتکل *SMTP* از سرور ایمیل مبدا و سرور ایمیل مقصد انتقال می‌ابد. در مقصد، گیرنده ایمیل به کمک پروتکل *IMAP = Internet mail access protocol* می‌تواند ایمیل دریافتی را بخواند و در سرور ایمیلش مرتب کند یا حذف کند. پروتکل *HTTP* به ما امکان *web mail* را می‌دهد مثل جیمیل و یاهو که درواقع یک *interface* وب ارائه می‌دهد که به کمک *SMTP* ایمیل را ارسال می‌کند و با *IMAP* یا *POP* ایمیل را دریافت *retrieve* می‌کند.

مراجع

[1] “ch2-v8-slide33to58,” <https://docs.google.com/presentation/>, 2022.